By using a background service to fetch the exchange rates, your API endpoint becomes even simpler. Instead of fetching the latest rates itself, it returns the value from the cache, which is kept up to date by the background service:

```
app.MapGet("/", (ExchangeRatesCache cache) =>
    cache.GetLatestRatesAsync());
```

This approach to caching works to simplify the API, but you may have noticed a potential risk: if the API receives a request before the background service has successfully updated the rates, the API will fail to return any rates.

This may be OK, but you could take another approach. As well as updating the rates periodically, you could use the `StartAsync` method to block app startup until the rates have successfully updated. That way, you guarantee that the rates are available before the app starts handling requests, so the API will always return successfully. Listing 34.4 shows how you could update listing 34.2 to block startup until the rates have been updated while still updating periodically in the background.

Listing 34.4 Implementing `StartAsync` to block startup in an `IHostedService`

```
public class ExchangeRatesHostedService : BackgroundService
{
    private readonly IServiceProvider _provider;
    private readonly ExchangeRatesCache _cache;
    public ExchangeRatesHostedService(
        IServiceProvider provider, ExchangeRatesCache cache)
    {
        _provider = provider;
        _cache = cache;
    }

    public override async Task StartAsync(                    ❶
        CancellationToken cancellationToken)                  ❶
```